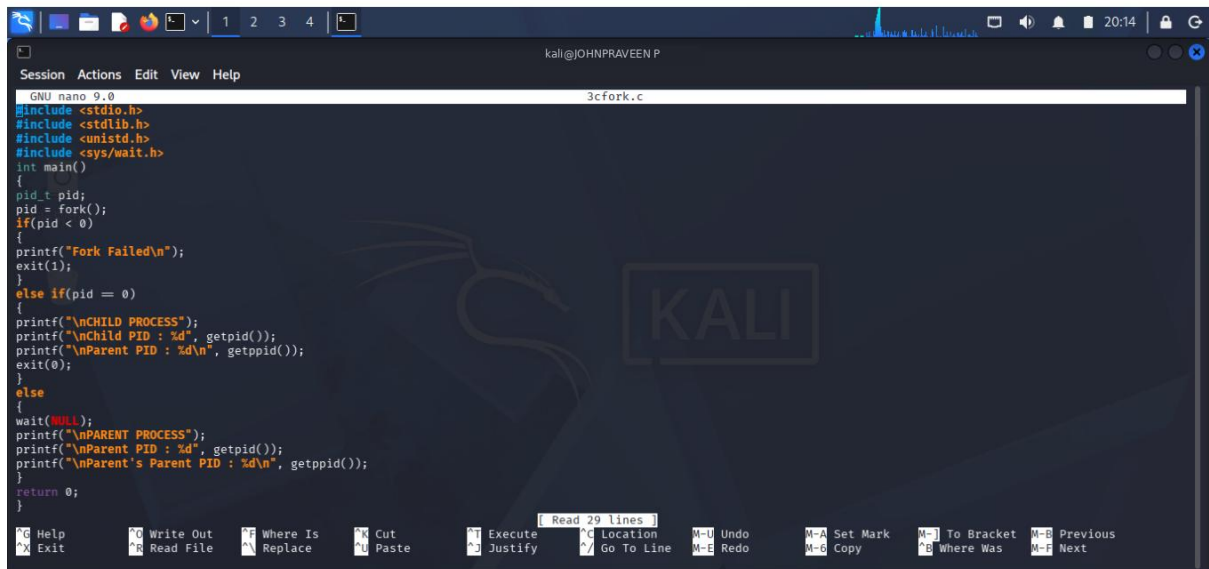
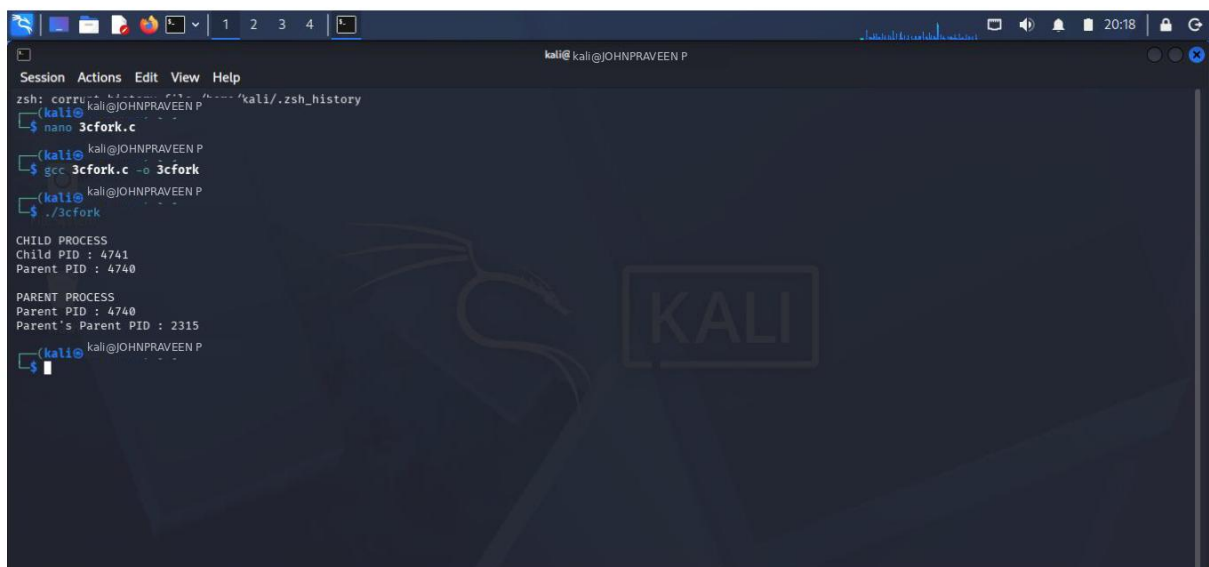


C PROGRAMMING : fork()



```
GNU nano 9.0 3cfork.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid < 0)
    {
        printf("Fork Failed\n");
        exit(1);
    }
    else if(pid == 0)
    {
        printf("\nCHILD PROCESS");
        printf("\nChild PID : %d", getpid());
        printf("\nParent PID : %d\n", getppid());
        exit(0);
    }
    else
    {
        wait(NULL);
        printf("\nPARENT PROCESS");
        printf("\nParent PID : %d", getpid());
        printf("\nParent's Parent PID : %d\n", getppid());
    }
    return 0;
}
```

OUTPUT :



```
zsh: corru... 'kali/.zsh_history'
(kali) kali@JOHNPRAVEEN P
$ nano 3cfork.c
(kali) kali@JOHNPRAVEEN P
$ gcc 3cfork.c -o 3cfork
(kali) kali@JOHNPRAVEEN P
$ ./3cfork

CHILD PROCESS
Child PID : 4741
Parent PID : 4740

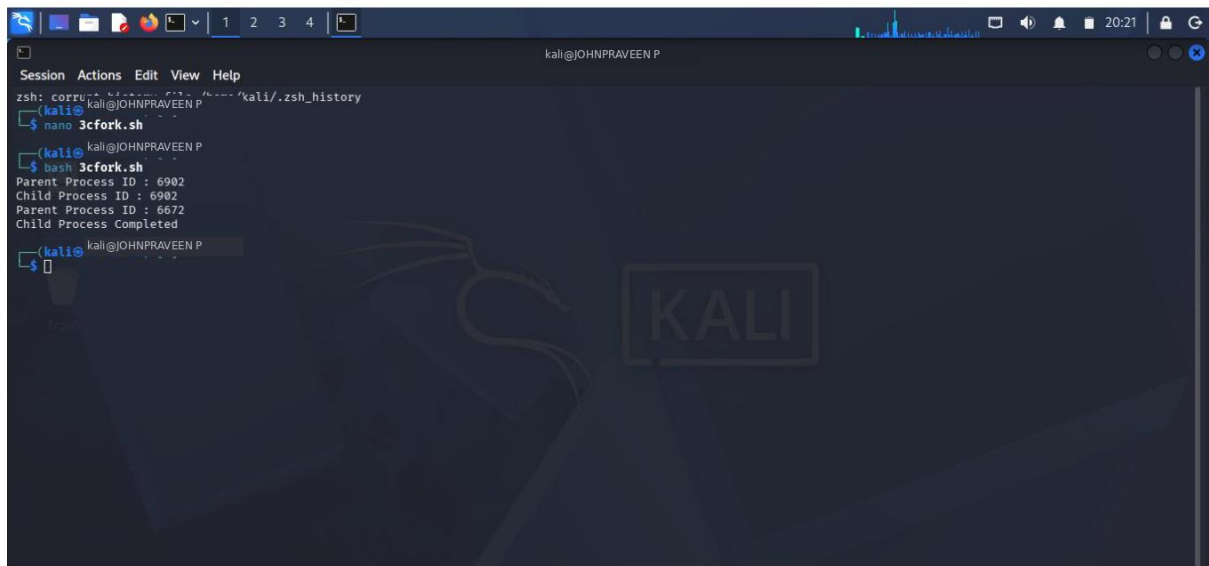
PARENT PROCESS
Parent PID : 4740
Parent's Parent PID : 2315
(kali) kali@JOHNPRAVEEN P
$
```

SHELL PROGRAMMING :fork()



```
GNU nano 9.0 3cfork.sh
#!/bin/bash
echo "Parent Process ID : $$"
(
  echo "Child Process ID : $$"
  echo "Parent Process ID : $PPID"
  exit 0
) &
wait
echo "Child Process Completed"
```

OUTPUT :



```
zsh: corrupt -- /home/kali/.zsh_history
kali@JOHNPRAVEEN P
$ nano 3cfork.sh
kali@JOHNPRAVEEN P
$ bash 3cfork.sh
Parent Process ID : 6902
Child Process ID : 6902
Parent Process ID : 6672
Child Process Completed
kali@JOHNPRAVEEN P
$
```

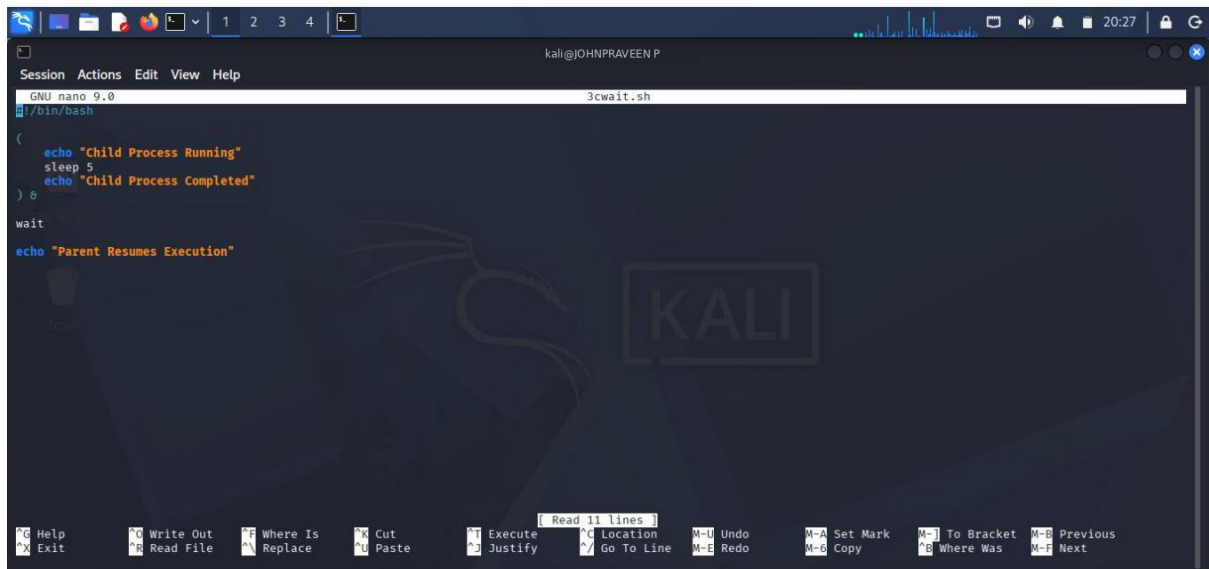
C PROGRAMMING :wait()

```
GNU nano 9.0 3cwait.c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    pid = fork();
    if(pid == 0)
    {
        printf("Child Process Running\n");
        sleep(5);
        printf("Child Process Completed\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Resumes Execution\n");
    }
    return 0; }
```

OUTPUT :

```
zsh: corrupt state - /home/kali/.zsh_history
kali@kali:~$ nano 3cwait.c
kali@kali:~$ gcc 3cwait.c -o 3cwait
kali@kali:~$ ./3cwait
Child Process Running
Child Process Completed
Parent Resumes Execution
kali@kali:~$
```

SHELL PROGRAMMING :wait()



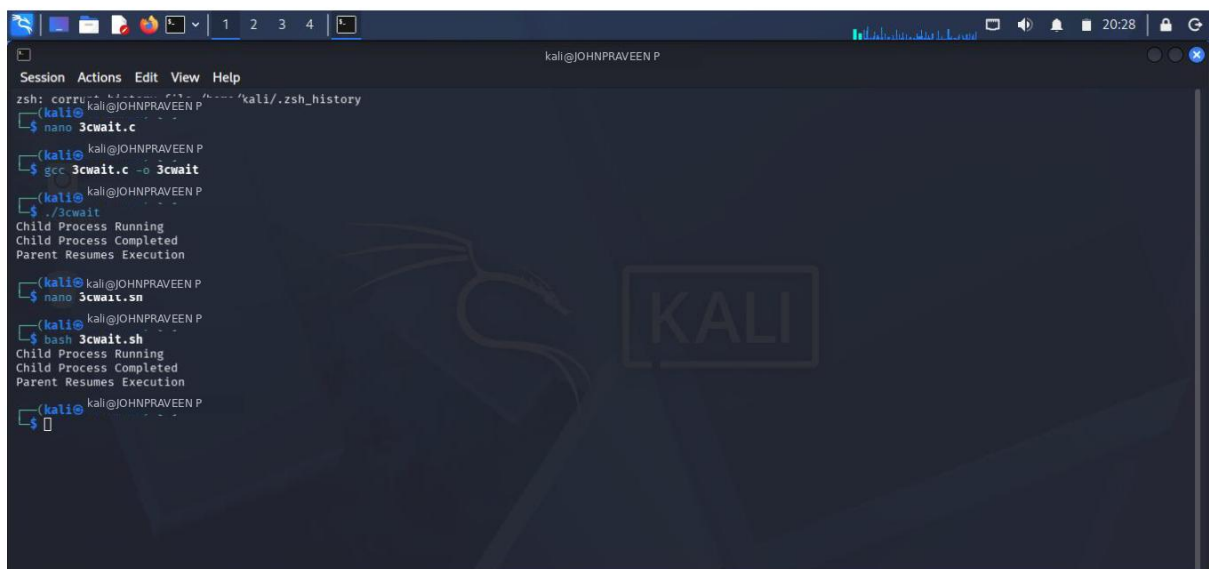
```
GNU nano 9.0 3cwait.sh
#!/bin/bash

(
    echo "Child Process Running"
    sleep 5
    echo "Child Process Completed"
) &

wait

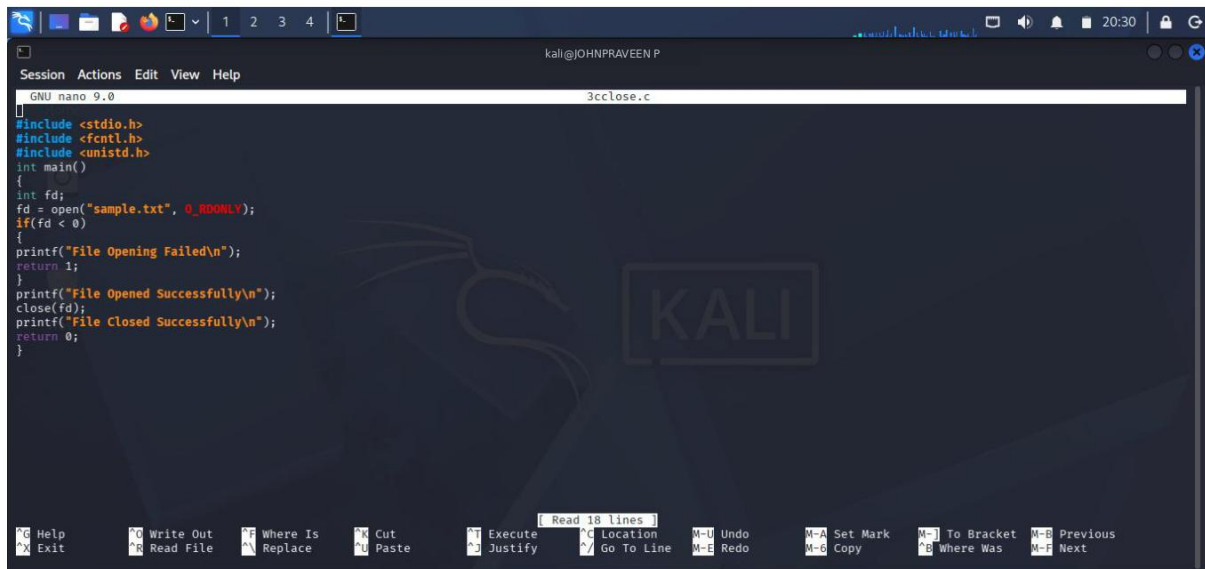
echo "Parent Resumes Execution"
```

OUTPUT :



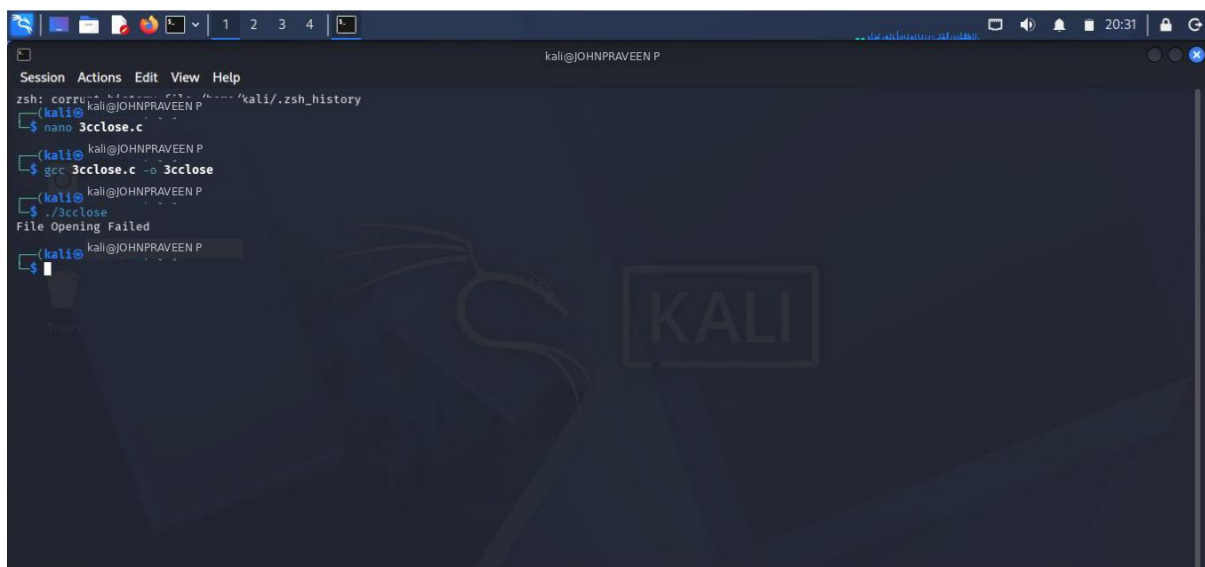
```
zsh: corrupt zsh_history file /home/kali/.zsh_history
kali@JOHNPRAVEEN P
$ nano 3cwait.c
kali@JOHNPRAVEEN P
$ gcc 3cwait.c -o 3cwait
kali@JOHNPRAVEEN P
$ ./3cwait
Child Process Running
Child Process Completed
Parent Resumes Execution
kali@JOHNPRAVEEN P
$ nano 3cwait.sh
kali@JOHNPRAVEEN P
$ bash 3cwait.sh
Child Process Running
Child Process Completed
Parent Resumes Execution
kali@JOHNPRAVEEN P
$
```

C PROGRAMMING :close()



```
GNU nano 9.0 3cclose.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int fd;
    fd = open("sample.txt", O_RDONLY);
    if(fd < 0)
    {
        printf("File Opening Failed\n");
        return 1;
    }
    printf("File Opened Successfully\n");
    close(fd);
    printf("File Closed Successfully\n");
    return 0;
}
```

OUTPUT :



```
zsh: corru... kali@JOHNPRAVEEN P ~ - ssh 'kali/.zsh_history'
(kali) kali@JOHNPRAVEEN P
$ nano 3cclose.c
(kali) kali@JOHNPRAVEEN P
$ gcc 3cclose.c -o 3cclose
(kali) kali@JOHNPRAVEEN P
$ ./3cclose
File Opening Failed
(kali) kali@JOHNPRAVEEN P
$
```

SHELL PROGRAMMING :close()

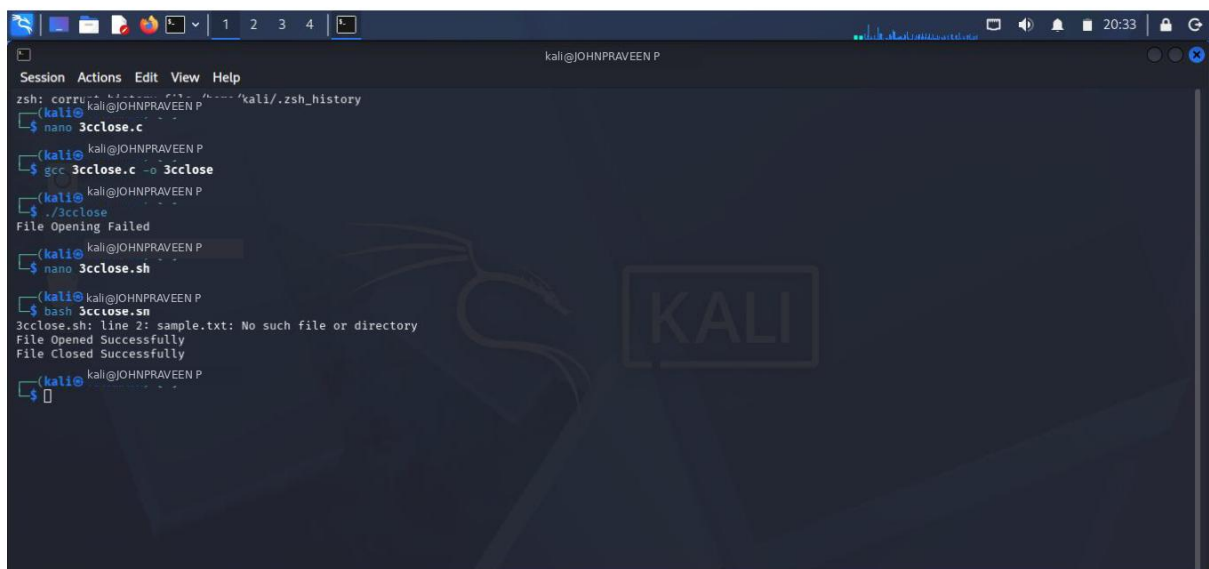


A screenshot of a terminal window on a Kali Linux system. The terminal is running the nano text editor, editing a file named `3cclose.sh`. The file's content is as follows:

```
#!/bin/bash
exec 3< sample.txt
echo "File Opened Successfully"
exec 3<&-
echo "File Closed Successfully"
```

The nano editor's status bar at the bottom shows various keyboard shortcuts for editing and execution. The background of the terminal features a faint Kali Linux logo.

OUTPUT :



A screenshot of a terminal window showing the execution of a shell script. The user runs `3cclose.c`, which compiles to `3cclose` and then runs it. The output shows that the file `sample.txt` does not exist, resulting in a "File Opening Failed" message. The user then runs `3cclose.sh`, which also fails with the same error message.

```
zsh: corru... 'kali/.zsh_history'
(kali) kali@JOHNPRAVEEN P
$ nano 3cclose.c
(kali) kali@JOHNPRAVEEN P
$ gcc 3cclose.c -o 3cclose
(kali) kali@JOHNPRAVEEN P
$ ./3cclose
File Opening Failed
(kali) kali@JOHNPRAVEEN P
$ nano 3cclose.sh
(kali) kali@JOHNPRAVEEN P
$ bash 3cclose.sh
3cclose.sh: line 2: sample.txt: No such file or directory
File Opened Successfully
File Closed Successfully
(kali) kali@JOHNPRAVEEN P
$
```